rollback_ux

Helix OTA — 1.0.1 End-User / Operator Rollback UX

Field	Value
Revision	2
Created	2026-06-08
Last modified	2026-06-08
Status	partially implemented (recall endpoint = forward-fix shipped)
Decision (2026-06-08, operator)	**Honor AVB — recall is forward-fix only.** A recall NEVER ships an image below the device's bootloader rollback index. `POST /deployments/{id}/recall` (IMPLEMENTED, `handlers_recall.go`) supersedes the current deployment and creates a NEW active deployment of the chosen target release; the update-check anti-downgrade invariant guarantees no device is offered a version ≤ its current, so AVB anti-rollback is honored by construction. A genuine sub-index downgrade is NOT offered (would require a deliberate operator-gated mechanism + matching rollback-index bump — out of scope). This resolves threat_model §11 item 11.
Status summary	The product/UX spec for **end-user / operator rollback in phase 1.0.1**. The operator ratified end-user rollback INTO 1.0.1 (2026-06-08), folding the former `1.0.2-rollback/` design reference into this phase. Specifies the three rollback trigger types (automatic A/B boot-failure — already an MVP guarantee; server/operator-driven recall to N-1; user-initiated device-local), the operator dashboard UX, the device-side/user-initiated path and its A/B

Field	Value
	single-previous-slot constraints, the REST surface (an operator **recall** endpoint extending the existing rollout API, writing a `rollback_history` row), the cardinal data-safety invariant (never leave a device unbootable / never touch `userdata`), and how rollback composes with staged-rollout halt/abort.
Scope	Reuses the **existing** migration-002 `rollback_history` table (`1.0.0-mvp/database/migrations/002_staged_rollout.up.sql`) — does NOT invent a new table. Extends the **existing** rollout API (`server/internal/api/handlers_rollout.go`, routes in `server/internal/api/server.go`) — does NOT redesign it.
Issues	Merge-window / boot-success timing values are UNVERIFIED on the Orange Pi 5 Max (see §10). Multi-version / true-downgrade is OUT of A/B single-previous-slot reach (deferred, §6). The persistent-store (pgx) recall path is gated on the migration-002 `StoragePort` landing (per phase README “REMAINING”).
Issues summary	Accept the rollback UX + recall endpoint shapes; do not bind UNVERIFIED timing constants; do not promise multi-version rollback.
Fixed	initial rollback-UX spec routed onto migration-002 `rollback_history` + the existing rollout API.
Fixed summary	Captured the three trigger types, the dashboard recall control, the user-initiated path predicates, the recall REST surface, the data-safety invariant, and the halt/abort composition — all grounded in real schema + real handlers, no invented tables/routes.
Continuation	Wire the recall handler + route once the migration-002 pgx `StoragePort` lands; confirm the merge-window / boot-success timing constants on a real Orange Pi 5 Max and drop the §10 UNVERIFIED tags; spec the device-side rollback-detection report path against `ota-android-agent`; four-layer test the recall authorization +

Field	Value
	data-safety paths; export HTML/PDF siblings per §11.4.65.

table_of_contents

> The ToC requirement is mandated by HelixConstitution §11.4.61 (UNVERIFIED clause number, carried per the MVP sibling specs). This document carries its ToC immediately after the metadata table.

---

§1. scope_and_provenance

This document specifies the **end-user / operator rollback experience** for phase **1.0.1**. Per the operator decision of 2026-06-08, end-user rollback is part of 1.0.1 (the `1.0.2-rollback/` directory is retained as a superseded design reference per §11.4.124, not deleted). This spec is the product/UX layer; it sits on top of, and routes onto, concrete existing assets:

- **Data backing — already landed.** The `rollback_history` table is in **migration 002** (`1.0.0-mvp/database/migrations/002_staged_rollout.up.sql`), real-DB validated. Its `kind` column is the closed set `{'abort','rollback'}`; its `kind_ref` CHECK requires a `'rollback'` row to carry both `from_release_id` and `to_release_id`, and an `'abort'` row to carry neither. **This spec reuses that table; it does NOT add a new one.**
- **API substrate — already landed.** The staged-rollout REST handlers (`server/internal/api/handlers_rollout.go`) and routes (`server/internal/api/server.go`, the `/api/v1/deployments/:deploymentId/rollout[...]` group) are real and role-gated. The recall endpoint in §7 **extends** that group; it does not replace it.
- **Automatic boot-failure rollback — already an MVP guarantee.** The AOSP A/B boot-control HAL + AVB auto-fallback is shipped in MVP. It is treated here as the substrate the human-driven paths sit on, **not** as new deliverable code (§3.1).

`triggered_by` on `rollback_history` is the FK to `helix_ota.users(id)` — the operator who authorized the rollback. For a purely automatic boot-failure event (no human authorizer) it is left NULL.

§2. rollback_vs_downgrade_distinction

Pinned so the UX never overpromises (source reference: `1.0.2-rollback/README.md` §1.3, re-based):

Term	Mechanism	Reach	Phase
Rollback	Instant A/B slot swap back to the immediate previous build (boot-control metadata change only)	Exactly **one** previous version	**1.0.1 (this spec)**
Downgrade	Full OTA cycle delivering an arbitrary older build as a new artifact	Any older version	Deferred — own sub-design (§6)

Everything in §3–§9 is **rollback**. The word "recall" is used for the *operator-driven fleet/single-device* form of rollback (it produces a server-side directive); "rollback" is the umbrella term and the `kind` value persisted.

§3. the_three_rollback_trigger_types

All three converge on the same A/B slot-swap substrate; they differ only in **who initiates** and **what audit row** results.

§3.1 automatic_ab_boot_failure (already MVP — substrate, not new code)

- **Path:** hardware/bootloader. The A/B boot-control HAL retry counter exhausts on the new slot → the bootloader falls back to the previous slot. AVB re-verifies the rolled-back slot.
- **New in 1.0.1 (report-after-the-fact only):** the device-side rollback-detection report — on the next *successful* boot the agent notices it is back on the pre-update slot and emits a `ROLLBACK` telemetry event. The server then writes a `rollback_history` row. For a pure boot-failure auto-fallback the natural `kind` is `'rollback'` (a real from→to version change occurred), with `triggered_by = NULL` (no human authorizer) and `details` carrying `{"origin":"automatic_boot_failure"}`. **UNVERIFIED:** whether the auto-fallback is recorded as `kind='rollback'` vs surfaced separately is a §10 open item — the migration-002 CHECK only admits `'abort'` and `'rollback'`.
- **UX:** none on the operator side beyond the audit/history surface (§4.3) and a device-health flag. The end user experiences a transparent recovery.

§3.2 server_operator_driven_recall_to_n_minus_1 (the primary new control-plane work)

- **Path:** an operator/admin invokes recall for a deployment (fleet) or a single device from the dashboard (§4) or directly via the recall endpoint (§7). The server classifies in-scope devices into:
  - **pre-merge** (Virtual A/B COW merge still pending) → slot-swappable, gets a rollback directive in its next update-check response;
  - **post-merge** (merge complete, previous slot reclaimed) → NOT slot-swappable → needs a downgrade artifact (OUT of scope, §6) → reported back to the operator as "not rollback-eligible";
  - **offline** → directive queued for next check-in.
- **Audit:** one `rollback_history` row, `kind='rollback'`, `from_release_id` = the regressing release, `to_release_id` = N-1, `triggered_by` = the operator, `deployment_id` = the recalled deployment, and `recall_deployment_id` = the new server-side recall deployment if one is materialized (the column exists in migration 002 precisely for this correlation).
- This is the **primary new deliverable** of the rollback work.

§3.3 user_initiated_device_local

- **Path:** an on-device Settings entry the end user can tap. Available **only** while the §5 predicates hold (bootable previous slot exists, Virtual A/B merge still pending, rollback window unexpired).
- **Audit:** the device reports the action via telemetry; the server writes a `rollback_history` row, `kind='rollback'`, `triggered_by = NULL` (the actor is the device end user, not a `users` row) with `details` carrying `{"origin":"user_initiated"}`.
- **UX:** §5.

§4. operator_dashboard_ux

The dashboard is the operator's surface; the device-side path (§5) is separate. The deployment screen gains a rollback/recall control.

§4.1 recall_control_on_the_deployment_screen

- On the deployment detail screen (the same screen that shows the staged-rollout phase cursor + health), add a **"Recall to previous version (N-1)"** action. It is visible only to `RoleOperator` / `RoleAdmin` (parity with the existing rollout-create role gate); `RoleViewer` sees the rollback **history** (§4.3) read-only but not the action.
- The control surfaces the eligibility split *before* the operator commits: a pre-flight (dry-run, §7) returns the count of rollback-eligible (pre-merge), not-eligible (post-merge), and queued (offline) devices, plus the resolved `from→to` releases. This is the "what will actually happen" preview — never a blind fire.
- **Scope toggle:** single-device recall vs whole-deployment (fleet) recall. Single-device is the low-blast-radius default surfaced first.

§4.2 confirmation

- A blocking confirmation dialog states, in plain language: the `from` version, the `to` (N-1) version, the affected device count (and the not-eligible count it will NOT touch), and the data-safety guarantee ("device user data is not modified; only the active boot slot is changed back", §8).
- The operator must explicitly confirm. Per §11.4.6 (no-guessing), the dialog shows resolved facts (counts, version ids) from the dry-run, never estimates.
- **Execution mode** is chosen here: **instant** (directive to all eligible devices at once) or **gradual / wave** (rollback delivered in cohorts, mirroring the staged-rollout phase shape) — the gradual form reuses the rollout engine's cohort logic rather than re-inventing it.

§4.3 audit_surface

- A **Rollback history** panel on the deployment screen lists `rollback_history` rows for that deployment (and a fleet-wide view elsewhere): timestamp, `kind` (`abort` vs `rollback`), from→to release, who (`triggered_by`, resolved to the user; "automatic" / "device user" when NULL with the `details.origin`), and `reason`.
- This panel is the operator-visible projection of the append-only audit table — it is read-only (the table has no UPDATE/DELETE path in normal operation; FKs are `ON DELETE SET NULL` so history outlives the releases/deployments it references).

§5. device_side_user_initiated_path

§5.1 availability_predicates (all must hold)

The Settings entry is shown/enabled only when ALL of:

1. a **bootable previous slot exists** (A/B inactive slot is intact);
2. the **Virtual A/B COW merge is still pending** (the previous slot has not been reclaimed — see the merge-window constraint, §8 + §9);
3. the **rollback window has not expired** (a bounded post-update window; default value is UNVERIFIED, §10).

When any predicate fails the entry is hidden or shown disabled-with-reason ("rollback no longer available: update finalized" / "rollback window expired") — never a button that fails after tap.

§5.2 confirmation_and_effect

- A device-local confirmation states the from→to version and the §8 data-safety guarantee ("your data and settings are kept").
- On confirm: the device sets the previous slot active via the boot-control HAL and reboots; it does **not** call `markBootSuccessful()` on the failed/new slot. On the next boot the device is on N-1; the rollback-detection report (§3.1 mechanism reused) emits the telemetry event that produces the `rollback_history` row.

§5.3 constraints

- A/B gives the device **exactly one** reachable previous slot. The user-initiated path can therefore only reach N-1; there is no on-device “pick an older version” list (that is multi-version, §6).
- The path is strictly **time-and-merge-bounded** (§5.1.2/§5.1.3): once the merge completes or the window expires, the previous slot is gone and the only remaining reversion is an operator-driven downgrade artifact (§6), not a device-local slot swap.

§6. multi_version_out_of_scope

A/B inherently supports exactly **one** previous version. Reaching an arbitrary older version (N-2, N-3, …) is **explicitly OUT of scope** for 1.0.1 rollback. It would require one of: additional slots (unsupported on RK3588 per the source reference), `/data`-stored images, or a server-side **downgrade** (full OTA cycle delivering the older build as a new artifact, §2). Multi-version rollback and delta-rollback are deferred to a later phase (the freed `1.0.2` slot / `1.0.3` delta-updates per the phase README), each needing its own downgrade-artifact + forward-migration + security-patch-level sub-design before being promised. The UX MUST NOT present a multi-version chooser in 1.0.1.

§7. rest_surface

The recall surface **extends** the existing rollout route group (`server/internal/api/server.go`, the `auth` group under `/api/v1`, alongside `POST /deployments/:deploymentId/rollout` and `/rollout/evaluate`). It does NOT introduce a new versioned prefix and follows the same handler/error/role conventions (`respondError` / `respondValidation` / `requireRole`).

§7.1 recall_endpoint

```
POST /api/v1/deployments/{deploymentId}/recall
  Roles: RoleOperator, RoleAdmin   (parity with rollout-create;
RoleViewer forbidden → 403)
  Audited: yes (auditMiddleware records the successful mutating
action)
```

Request body (proposed wire type, mirroring the `RolloutCreate`/`RolloutVerdict` style already in `handlers_rollout.go`):

```
{
  "scope":   "deployment | device",
  "device_id": "<uuid, required when scope=device>",
  "mode":    "instant | gradual",
  "dry_run": false,
  "reason":  "regression in phase 2 cohort"
}
```

Behavior:

- **404** when the deployment does not exist (same `GetDeployment` precheck as `handleCreateRollout`).
- **400** when the body is malformed, `scope`/`mode` is outside its closed set, or `scope=device` without `device_id` (`respondValidation` with an `ErrorDetail`).
- **`dry_run=true`** returns the eligibility split (pre-merge eligible / post-merge not-eligible / offline queued) + resolved `from`/`to` release ids and writes **no** `rollback_history` row. This backs the §4.1 pre-flight.
- **`dry_run=false`** resolves N-1 (the `to` release), classifies devices, injects the rollback directive (instant or gradual), and writes exactly **one** `rollback_history` row: `kind='rollback'`, `from_release_id` = current deployment release, `to_release_id` = N-1, `triggered_by` = the JWT subject's user id, `deployment_id` = `{deploymentId}`, `recall_deployment_id` = the materialized recall deployment (if any), `reason` = the request `reason`, `details` = `{"origin":"operator_recall","scope":...,"mode":...,"counts":{...}}`.
- Returns the resulting rollback summary (eligible/queued/skipped counts + the `rollback_history` row id). The rollout `status` for the affected deployment moves to the `rolled_back` control-plane overlay (one of the migration-002 `rollouts.status` overlay values, alongside `paused`/`aborted`).

§7.2 history_read

```
GET /api/v1/deployments/{deploymentId}/rollbacks
  Roles: RoleViewer, RoleOperator, RoleAdmin   (read parity with
GET .../rollout)
```

Returns the `rollback_history` rows for the deployment (both `kind='abort'` and `kind='rollback'`), newest first — the data behind the §4.3 audit panel.

> Implementation note (honest status): the route + handler are **not yet wired**. The phase README lists the pgx `StoragePort` over migration-002 tables as REMAINING; the recall handler depends on persistent reads/writes of `rollback_history`, so it lands with that store. The in-memory rollout `Service` (`server/internal/rollout/service.go`) has no rollback method today. This is a spec of the surface, not a claim that it ships.

§8. data_safety_invariant

The cardinal invariant, non-negotiable (Constitution §9 absolute data safety; source reference `1.0.2-rollback/README.md` §5):

1. **Never leave a device unbootable.** A rollback only ever makes an **already-bootable, already-verified** previous slot active. It never erases, reflashes, or partially writes the slot the device is currently running from. If the previous slot is not intact + bootable, the device is **not** rollback-eligible (it falls into the post-merge / not-eligible class, §3.2) — the operation is refused, never attempted half-way.
2. **Never touch `userdata`.** A slot swap changes **only** boot-control metadata. `/data` (user data, settings, app data) is untouched. The operator and device-local

confirmation copy both state this explicitly (§4.2, §5.2). This MUST be pinned by a pre/post integrity test (userdata hash unchanged across a rollback) on real hardware.
3. **Authorized rollback ≠ malicious downgrade.** The rollback authorization (operator JWT / device-local user action against an intact previous slot) must be distinguishable from a malicious downgrade offer; the AVB rollback-index / anti-downgrade invariant must be preserved, not defeated (1.0.1 §5 / synthesis G1). An authorized N-1 rollback to an already-installed, already-index-valid slot does not violate anti-downgrade because that slot was the running build moments earlier.

§9. composition_with_staged_rollout_halt_abort

Rollback is **distinct from** the staged-rollout engine's `halt`/`abort`, and the two are reconciled at the `rollback_history.kind` and `rollouts.status` layers:

- **abort** (`kind='abort'`) — the staged-rollout engine's existing safety action: a regressing cohort's rollout is **stopped** (no further devices receive the new build). Per the migration-002 CHECK, an `'abort'` row carries **no** from/to release — nothing was reverted, advancement was merely halted. The engine's automatic halt-on-error-breach (`handleEvaluateRollout` → `halt` decision) and the operator one-click abort both produce `kind='abort'`. Abort moves `rollouts.status` to the `aborted` overlay.
- **rollback** (`kind='rollback'`) — actually **reverts already-updated devices** to N-1 (§3.2/§3.3). Carries both from/to releases. Moves `rollouts.status` to the `rolled_back` overlay.
- **Ordering / safety-invariant alignment:** halt/abort wins over advance in a single evaluation window (1.0.1 rollout-engine safety invariant). A recall (`rollback`) is naturally preceded by an abort: the operator first stops the bleeding (abort → no new devices get the bad build) and then recalls the already-updated cohort (rollback → revert them). The dashboard MAY offer "abort + recall" as one operator gesture that produces **two** audit rows (one `abort`, one `rollback`), preserving the closed-set semantics rather than overloading a single row.
- **Merge-window dependency on 1.0.1's health window.** Slot-swap rollback is possible **only before** the Virtual A/B COW merge completes. The device therefore delays `markBootSuccessful()` until the 1.0.1 post-boot health-confirmation window passes — rollback **depends on** that 1.0.1 health-gating mechanism rather than re-inventing it. This is the load-bearing timing edge between staged rollout and rollback.

§10. anti_bluff_unverified_register

Per Constitution §11.4.6 (no-guessing) / §11.4 (no-bluff) — MUST NOT propagate as fact:

- **Rollback window default (e.g. 7 days), boot-success / merge-delay timing, retry-counter values** — UNVERIFIED on RK3588 / Orange Pi 5 Max / the shipped Android version. Source-doc defaults only; confirm on real hardware before binding any constant.

- **Virtual A/B merge behavior under mid-merge rollback** (`snapuserd`/ bootloader) — UNVERIFIED on the Orange Pi 5 Max; hardware-gated spike required before §5.1.2 / §8.1 are asserted as facts.
- **Whether automatic boot-failure auto-fallback is persisted as `kind='rollback'` vs surfaced via a separate signal** — UNVERIFIED; migration-002's `rollback_history_kind_chk` admits only `{'abort','rollback'}`, so §3.1's choice (`kind='rollback'`, `triggered_by=NULL`) is a proposal pending the device-side report spec, not a landed decision.
- **dm-verity / verified-boot-state transitions on rollback** — UNVERIFIED on RK3588.
- **The recall endpoint + history read (§7) are NOT yet wired** — they depend on the migration-002 pgx `StoragePort` (phase README "REMAINING"). This document specifies the surface; it does not claim shipped behavior. No `rollout.Service` rollback/recall method exists today (verified against `server/internal/rollout/service.go`).
- **The dashboard is a server-side product surface not present in this repo at spec time** — §4 is a UX specification, not a description of existing UI (UNVERIFIED that any dashboard front-end exists yet).
- **HelixConstitution §11.4.x clause numbers** cited here remain UNVERIFIED except those confirmed in `tests/test_strategy.md` §13 (carried from the MVP sibling specs).

§11. sources

- `1.0.1-staged-rollout/README.md` (this phase; §5 End-user rollback, the operator's INTO-1.0.1 ratification).
- `1.0.2-rollback/README.md` (SUPERSEDED design reference — rollback trigger types, merge-window constraint, data-safety invariant, rollback-vs-downgrade table; folded here).
- `1.0.0-mvp/database/migrations/002_staged_rollout.up.sql` (the real `rollback_history` table reused by this spec — `kind` closed set, `kind_ref` CHECK, FK survivability; `rollouts.status` overlays).
- `server/internal/api/handlers_rollout.go` + `server/internal/api/server.go` (the existing rollout REST handlers/routes/roles the recall surface extends).
- `server/internal/rollout/service.go` (the control-plane facade — confirms no rollback method exists yet; honest-status anchor for §7).